

Evaluating IoT protocols with the 6TiSCH simulator

The objective of this TP is to use 6TiSCH, an open simulator, for evaluating specific IoT protocols and algorithms. We are going to start by downloading, installing and getting familiar with how 6TiSCH works. We will then use it to evaluate the impact of the cell scheduling algorithm on IoT networks that use the IEEE 802.15.4 as the radio technology.

1. What is the 6TiSCH simulator

The 6TiSCH simulator is an open-source, event-driven simulator written in Python. Its objective is to provide a light-weight tool for evaluating the 6TiSCH stack. As a result, it makes no effort to model the physical link or the hardware but focuses on the MAC layer and above.

The 6TiSCH stack is a protocol suite proposed by the 6TiSCH working group at the IETF [1]. It aims at proposing an IPv6-based IoT stack using IEEE 802.15.4. As we saw in lecture, IEEE 802.15.4 does not define a scheduling algorithm, so one of the contributions of the 6TiSCH working group is to define and standardize such an algorithm [2], based on ORCHESTRA, which we saw in lecture. In addition, it introduces a routing algorithm, RPL [3], and mechanisms for compressing IPv6 headers, necessary for IoT, which you will see in the IoT Interconnection course.

2. Downloading and installing the 6TiSCH simulator

<https://bitbucket.org/6tisch/simulator/src/master/>

The above link gives detailed instructions for how to download and install the 6TiSCH simulator.

3. Getting familiar with the 6TiSCH simulator

After installing the 6TiSCH simulator, to run the first simulation it suffices to enter the bin directory and type

```
$python runSim.py
```

As every simulator, the 6TiSCH simulator generates a trace file which then needs to be processed for generating different KPIs. In addition, this simulator goes a bit further because it includes already python scripts for processing the data and automatically generating graphs.

Every time you run a simulation, the 6TiSCH simulator creates a new directory in the directory bin/simData. The directory includes the raw trace file, the file in *.dat format as well as graphs for certain common KPIs, including latency, network lifetime, etc.

To interact with the simulation engine, the 6TiSCH simulator uses the config.json file in the bin directory. It allows us to configure things like simulation time, change specific protocol parameters, select which cell scheduling algorithm to use for the simulation, etc.

GUI: Finally, the 6TiSCH simulator provides a graphical user interface for running simulations. To use it, change to the gui directory and run backend/start, then go to <http://127.0.0.1:8080> using your web browser.

4. Evaluating TSCH scheduling using the 6TiSCH simulator

As we saw in lecture, the algorithm for allocation the time/frequency slots of the TSCH, called cells, is a very important part of an IoT network based on IEEE 802.15.4. Using the the 6TiSCH simulator, you will evaluate the impact of MSF [2], the algorithm introduced by the 6TiSCH working group. For context, we will compare it to SFNone, a trivial algorithm that gives all IoT nodes all the slots, therefore, reducing channel access to CSMA.

To run a simulation with a specific scheduling algorithm, you need to edit the file config.json. In line 35 you'll find the field "sf_class", which by default is assigned the value "SFNone". To run a simulation with MSF, you need to simply change this value to MSF. Alternatively, you can use the gui which allows to compare the two algorithms in terms of packet delivery rate (PDR), latency and slot occupancy.

References

- [1] <https://datatracker.ietf.org/wg/6tisch/about/>
- [2] <https://tools.ietf.org/html/draft-ietf-6tisch-msf-18>
- [3] <https://tools.ietf.org/html/rfc6550>